SEMESTER S5

MICROCONTROLLERS AND EMBEDDED SYSTEMS

Dr. MAYADEVI N CET

Syllabus

Module No.	Syllabus Description	Contact Hours
1	**Introduction to Microprocessors and Microcontrollers**-comparison of microprocessors and microcontrollers-Introduction to Embedded Systems-Application domain of embedded systems, features and characteristics, hard and soft real time systems **8051-Microcontrollers Hardware:** Microcontroller Architecture: IO Port structure, Register organization, general purpose RAM, Bit Addressable RAM, Special Function Registers (SFRs).	9
2	**Assembly programming of 8051:** Introduction to8051 assembly programming, Data types and Assembler directives, 8051 Addressing Modes, simple Assembly language programs(data transfer and arithmetic operations only) **8051 programming in C:** Data types and time delay in 8051, I/O	11

Syllabus

	programming in 8051 **8051 Timer/Counter programming in embedded C:** Programming 8051 timers, Counter programming,	
3	**8051 serial port programming in embedded C:** Basics of serial communication, 8051 connections to RS232, serial port programming in 8051. **8051 Interrupt programming in embedded C:** 8051 interrupts, external hardware and serial communication interrupt, Interrupt priority in 8051, Interrupt programming in C. **Interfacing:** LCD, ADC & DAC. Motor control: Relays and, stepper motor interfacing, DC motor interfacing and PWM using 8051	**12**
4	**Introduction to ARM processors** –ARM core-ARM Microcontroller-RISC vs CISC-Advanced features of ARM-Architecture versions-ARM Architecture-Instruction set architecture, operating modes, register set, General purpose registers- mode switching, conditional flags, Simple ALP programs on Arithmetic & logical operation, addition, subtraction, multiplication, division and factorial.	**12**

8051 Programming in C

- Why Program 8051 in ALP?
- Compilers produce hex file that is downloaded to ROM of microcontroller.
- The size of hex file generated by the compiler is the main concern, for two reasons:
- Microcontrollers have limited on-chip ROM
- Code space for 8051 is limited to 64K bytes
- The ALP language produces a hex file that is much smaller than C.
- **Also** Faster execution and direct hardware control

8051 Programming in C

- Why Program 8051 in C?
- The reasons for writing programs in C instead of assembly are:
- It is **easier and less time consuming** to write in C than Assembly
- C is **easier to modify and update**
- **Function libraries** are available for use
- C code is portable to other microcontroller with little or no modification

Data types in 8051 C

- The different data types are:
  - unsigned char
  - signed char
  - unsigned int
  - signed int
  - sbit (single bit)
  - bit and sfr

Data types in 8051 C

Table 1: Widely used 8051 C data types

Data Type	Size in Bits	Data Range/Usage
unsigned char	8-bit	0 to 255
(signed) char	8-bit	-128 to +127
unsigned int	16-bit	0 to 65535
(signed) int	16-bit	-32768 to +32767
sbit	1-bit	SFR bit-addressable only
bit	1-bit	RAM bit-addressable only
sfr	8-bit	RAM addresses 80 – FFH only

Data types in 8051 C

unsigned char:

- The unsigned char is an 8-bit data type takes a value in the range of 0– 255 (00H– FFH) **and the most widely used data type for 8051.**
- can be used in situations such **as setting a counter value and ASCII characters, instead of signed char.**

Data types in 8051 C

- **unsigned int**:
- The unsigned int is a 16-bit data type that takes a value in the range of 0 to 65535(0000 FFFFH).
- This is used to define **16-bit variables such as memory addresses, set counter values of more than 256.**

Data types in 8051 C

- **signed int:**
- **Signed int** is a 16-bit data type that uses the MSB D15 to represent–or +value.
- Have 15 bits for the magnitude of the number from–32768 to +32767.

Data types in 8051 C

- sbit(Single bit):
- The 8 bit keyword is a widely used 8051 C data types which is used to access single-bit addressable register.
- It allows access to the single bits of the SFR registers.
- Some of the SFRs are bit addressable.
- Among the SFRs that are widely used are also bit addressable ports P0-P3

Data types in 8051 C

- bit and sfr: The bit data type allows access to single bits of bit-addressable memory spaces **20– 2FH.**
- To access the byte-size SFR registers, use the sfr data type.

Embedded C program

Write an 8051 C program to toggle bits of P1 ports continuously with a 250 ms.

Solution:

```
#include <reg51.h>
void MSDelay(unsigned int);
void main(void)
  {
    while (1)                    //repeat forever
       {
         p1=0x55;
         MSDelay(250);
         p1=0xAA;
         MSDelay(250);
       }
  }

void MSDelay(unsigned int itime)
  {
    unsigned int i,j;
    for (i=0;i<itime;i++)
       for (j=0;j<1275;j++);
  }
```

Embedded C program

Example 2: Write an 8051 C program to send hex values for ASCII characters of 0, 1, 2, 3, 4, 5, A, B, C, and D to port P1.

Solution:

```
#include <reg51.h>
void main(void)
{
        unsigned char mynum[]="012345ABCD";
        unsigned char z;
        for (z=0;z<=10;z++)
        P1=mynum[z];
}
```

Write an 8051 C program to get a byte of data form P1, wait 1/2 second, and then send it to P2.

Solution:

```
#include <reg51.h>
void MSDelay(unsigned int);

void main(void)
  {
    unsigned char mybyte;
    P1=0xFF;                    //make P1 input port
    while (1)
       {
         mybyte=P1;             //get a byte from P1
         MSDelay(500);
         P2=mybyte;             //send it to P2
       }
  }
```

Write an 8051 C program to get a byte of data form P0. If it is less than 100, send it to P1; otherwise, send it to P2.

Solution:

```
#include <reg51.h>

void main(void)
  {
    unsigned char mybyte;
    P0=0xFF;                    //make P0 input port
    while (1)
       {
         mybyte=P0;             //get a byte from P0
         if (mybyte<100)
            P1=mybyte;          //send it to P1
         else
            P2=mybyte;          //send it to P2
       }
  }
```

Bit Level Programming

Write an 8051 C program to toggle only bit P2.4 continuously without disturbing the rest of the bits of P2.

Solution:

```
//Toggling an individual bit
#include <reg51.h>
sbit mybit=P2^4;

void main(void)
  {
    while (1)
       {
         mybit=1;          //turn on P2.4
         mybit=0;          //turn off P2.4
       }
  }
```

Ports P0 – P3 are bit-addressable and we use *sbit* data type to access a single bit of P0 - P3

Use the Px^y format, where x is the port 0, 1, 2, or 3 and y is the bit 0 – 7 of that port

A door sensor is connected to the P1.1 pin, and a buzzer is connected to P1.7. Write an 8051 C program to monitor the door sensor, and when it opens, sound the buzzer. You can sound the buzzer by sending a square wave of a few hundred Hz.

Solution:

```
#include <reg51.h>
void MSDelay(unsigned int);
sbit Dsensor=P1^1;
sbit Buzzer=P1^7;

void main(void)
  {
    Dsensor=1;                  //make P1.1 an input
    while (1)
       {
         while (Dsensor==1)//while it opens
           {
              Buzzer=0;
              MSDelay(200);
              Buzzer=1;
              MSDelay(200);
           }
       }
  }
```

The data pins of an LCD are connected to P1. The information is latched into the LCD whenever its Enable pin goes from high to low. Write an 8051 C program to send "The Earth is but One Country" to this LCD.

Solution:

```
#include <reg51.h>
#define LCDData P1   //LCDData declaration
sbit En=P2^0;        //the enable pin

void main(void)
  {
    unsigned char message[]
              ="The Earth is but One Country";
    unsigned char z;
    for (z=0;z<28;z++) //send 28 characters
       {
         LCDData=message[z];
         En=1;     //a high-
         En=0;     //-to-low pulse to latch data
       }
  }
```

Write an 8051 C program to get bit P1.0 and send it to P2.7 after inverting it.

Solution:

```
#include <reg51.h>
sbit inbit=P1^0;
sbit outbit=P2^7;
bit membit;

void main(void)
  {
    while (1)
       {
         membit=inbit;    //get a bit from P1.0
         outbit=~membit; //invert it and send
                          //it to P2.7
       }
  }
```

LOGIC OPERATIONS Bit-wise Operators in C

- Logical operators
  - AND (&&), OR (||), and NOT (!)
- Bit-wise operators
  - AND (&), OR (|), EX-OR (^), Inverter (~), Shift Right (>>), and Shift Left (<<)
    - These operators are widely used in software engineering for embedded systems and control

Bit-wise Logic Operators for C

		AND	OR	EX-OR	Inverter
A	B	A&B	A|B	A^B	~B
0	0	0	0	0	1
0	1	0	1	1	0
1	0	0	1	1	
1	1	1	1	0	